

Compiler_Project2_Parser

2019060546

주원웅

1. 컴파일

```
juwowa@juwowa: ~/loucomp_parse
# Makefile for C-Minus
#
# ./lex/tiny.l      --> ./cminus.l (from Project 1)
# ./yacc/tiny.y     --> ./cminus.y
# ./yacc/globals.h  --> ./globals.h

CC = gcc

CFLAGS = -W -Wall

OBSJ = main.o util.o lex.yy.o y.tab.o

.PHONY: all clean
all: cminus_parser

clean:
    rm -vf cminus_parser *.o lex.yy.c y.tab.c y.tab.h y.output

cminus_parser: $(OBSJ)
    $(CC) $(CFLAGS) $(OBSJ) -o $@ -lfl

main.o: main.c globals.h util.h scan.h parse.h y.tab.h
    $(CC) $(CFLAGS) -c main.c

util.o: util.c util.h globals.h y.tab.h
    $(CC) $(CFLAGS) -c util.c

scan.o: scan.c scan.h util.h globals.h y.tab.h
    $(CC) $(CFLAGS) -c scan.c

lex.yy.o: lex.yy.c scan.h util.h globals.h y.tab.h
    $(CC) $(CFLAGS) -c lex.yy.c

lex.yy.c: cminus.l
    flex cminus.l

y.tab.h: y.tab.c

y.tab.o: y.tab.c parse.h
    $(CC) $(CFLAGS) -c y.tab.c

y.tab.c: cminus.y
    yacc -d -v cminus.y

~
"Makefile" 43L, 843B
1,1 All
87% 10/29, 11:00 AM non
```

```
juwowa@juwowa: ~/loucomp_parse
juwowa@juwowa:~/loucomp_parse$ make
yacc -d -v cminus.y
gcc -W -Wall -c main.c
main.c:48:1: warning: return type defaults to 'int' [-Wimplicit-int]
  48 | main(int argc, char *argv[])
      | ^~~~
gcc -W -Wall -c util.c
util.c:192:8: warning: type defaults to 'int' in declaration of 'indentno' [-Wimplicit-int]
  192 | static indentno = 0;
      |          ^~~~~~
flex cminus.l
gcc -W -Wall -c lex.yy.c
lex.yy.c:1282:17: warning: 'yyunput' defined but not used [-Wunused-function]
 1282 |     static void yyunput (int c, char * yy_bp )
      |                   ^~~~~~
gcc -W -Wall -c y.tab.c
gcc -W -Wall main.o util.o lex.yy.o y.tab.o -o cminus_parser -lfl
juwowa@juwowa:~/loucomp_parse$
```

2. 개발 환경

- Ubuntu 22.04

3. 구현과 작동

이번 프로젝트에서는 main.c, globals.h, utils.c 의 일부 코드를 수정한 후, cminus.y 파일에 파싱 룰을 작성하여 파서를 구현하였습니다.

- 1) Main.c 수정부분

```
#define NO_PARSE FALSE
/* set NO_ANALYZE to TRUE
#define NO_ANALYZE TRUE

/* allocate and set tracing
int EchoSource = FALSE;
int TraceScan = FALSE;
int TraceParse = TRUE;
int TraceAnalyze = FALSE;
int TraceCode = FALSE;

int Error = FALSE;
```

- Parse 에 필요한 부분들을 수정해주었습니다.

- 2) **Globals.h** 수정부분

```
typedef enum
{
    DeclareK,
    StmtK,
    ExpK,
} NodeKind;
```

```
typedef enum
{
    ArrayDK,
    NonArrayDK,
    FuncDK,
    ParamDK,
    NonParamDK
} DeclareKind;
```

```
typedef enum
{
    CompK,
    IterK,
    IfElseK,
    IfK,
    NonValRetK,
    ValRetK
} StmtKind;
```

```
typedef enum
{
    ArrayDK,
    NonArrayDK,
    FuncDK,
    ParamDK,
    NonParamDK
} DeclareKind;
```

```
typedef struct treeNode
{
    struct treeNode *child[MAXCHILDREN];
    struct treeNode *sibling;
    int lineno;
    NodeKind nodekind;
    union
    {
        StmtKind stmt;
        ExpKind exp;
        DeclareKind dc;
    } kind;
    union
```

- 구현에 필요한 **Node** 의 타입들을 추가하여 만들고, treeNode 타입에 union 을 통해세가지 중 하나만 사용되도록 해주었습니다.
- 추가로 Make 시 충돌을 제거하기 위해 **#define ENDFILE 0** 부분을 삭제하였습니다.

```
juwowa@juwowa: ~/loucomp_parse
globals.h:35: warning: "ENDFILE" redefined
 35 | #define ENDFILE 0
    |
In file included from globals.h:30,
                from util.c:9:
y.tab.h:124: note: this is the location of the previous definition
124 | #define ENDFILE 287
    |
util.c:192:8: warning: type defaults to 'int' in declaration of 'indentno' [
 85% 10/29, 11:05 AM yyunput
```

- 3) util.c 수정부분

TreeNode * newDeclareNode(DeclareKind);

- Util.h 에 추가할 함수를 선언해주었습니다.

```
TreeNode *newDeclareNode(DeclareKind kind)
{
    TreeNode *t = (TreeNode *)malloc(sizeof(TreeNode));
    int i;
    if (t == NULL)
        fprintf(listing, "Out of memory error at line %d\n", lineno);
    else
    {
        for (i = 0; i < MAXCHILDREN; i++)
            t->child[i] = NULL;
        t->sibling = NULL;
        t->nodekind = DeclareK;
        t->kind.dc = kind;
        t->lineno = lineno;
    }
    return t;
}
```

- 추가가 필요한 declare node 의 생성함수를 작성하였습니다.

```
void printTree(TreeNode *tree)
{
    int i;
    INDENT;
    while (tree != NULL)
    {
        printSpaces();
        if (tree->nodekind == StmtK)
        {
            switch (tree->kind.stmt)
            {
                case CompK:
                    fprintf(listing, "Compound Statement:\n");
                    break;
                case IfElseK:
                    fprintf(listing, "If-Else Statement:\n");
                    break;
                case IfK:
                    fprintf(listing, "If Statement:\n");
                    break;
                case IterK:
                    fprintf(listing, "While Statement:\n");
                    break;
                case ValRetK:
                    fprintf(listing, "Return Statement:\n");
                    break;
                case NonValRetK:
                    fprintf(listing, "Non-value Return Statement:\n");
                    break;
                default:
                    fprintf(listing, "Unknown ExpNode kind\n");
                    break;
            }
        }
        else if (tree->nodekind == ExprK)
        {
            switch (tree->kind.exp)
            {
                case AssignK:
                    fprintf(listing, "Assign:\n");
                    break;
                case BinK:
                    fprintf(listing, "Op: ");
                    printToken(tree->attr.op, "\\0\n");
                    break;
                case ConstK:
                    fprintf(listing, "Const: %d\n", tree->attr.val);
                    break;
                case IdK:
                    fprintf(listing, "Variable: name = %s\n", tree->attr.name);
                    break;
                case CallK:
                    fprintf(listing, "Call: function name = %s\n", tree->attr.name);
                    break;
                default:
                    fprintf(listing, "Unknown ExpNode kind\n");
                    break;
            }
        }
        else if (tree->nodekind == DeclareK)
        {
            switch (tree->kind.dc)
            {
                case ArrayDK:
                    fprintf(listing, "Variable Declaration: name = %s, type = %s\n", tree->attr.name, tree->type ? "int[]" : "void[]");
                    break;
                case NonArrayDK:
                    fprintf(listing, "Variable Declaration: name = %s, type = %s\n", tree->attr.name, tree->type ? "int" : "void");
                    break;
                case FuncDK:
                    fprintf(listing, "Function Declaration: name = %s, return type = %s\n", tree->attr.name, tree->type ? "int" : "void");
                    break;
                case ParamDK:
                    fprintf(listing, "Parameter: name = %s, type = %s\n", tree->attr.name, tree->type ? "int" : "void");
                    break;
                case NonParamDK:
                    fprintf(listing, "Void Parameter\n");
                    break;
                default:
                    fprintf(listing, "Unknown ExpNode kind\n");
                    break;
            }
        }
        tree = tree->sibling;
    }
}
```

- 각각 Node 에 대해 알맞는 print 를 작성해주었습니다.

- 4) **cminus.y** 수정부분

```
zfgfjc 꺾꺾 λλeλλoλ(cμgλ * wεzεgδe):  
zfgfjc 꺾꺾 λλjεx(λo꺾q): \\ gqgεq JJ\J\JJ fo εuεnλε uo couλj꺾c w꺾꺾 jεx
```

- 파서 파일에서 함수 이름 충돌을 피하기 위한 방어 코드 추가 입니다.

```
%}  
%nonassoc IFX  
%nonassoc ELSE  
  
%token IF ELSE WHILE RETURN INT VOID  
%token ID NUM  
  
%left COMMA  
%right ASSIGN  
%nonassoc LT LE GT GE EQ NE  
%left PLUS MINUS  
%left TIMES OVER  
%left LPAREN RPAREN LBRACE RBRACE LCURLY RCURLY SEMI  
%token ERROR ENDFILE
```

- If-else 구문에서 dangling else problem 을 해결하기 위해서
%nonassoc IFX
%nonassoc ELSE 를 통해 결합성을 없애고 우선순위를 부여하였습니다.
- 연산자 우선순위를 지정해주었습니다.

```

program : declare_list
        { savedTree = $1; }
;

declare_list : declare_list declare {
    YYSTYPE t = $1;
    if(t!=NULL){
        while(t->sibling != NULL)
            t=t->sibling;
        t->sibling = $2;
        $$ = $1;
    }
    else{
        $$ = $2;
    }
}
| declare {
    $$ = $1;
}
;

declare : var_declare{$$ = $1; } | func_declare{$$ = $1;};
var_declare : type_spec id SEMI {
    $$ = newDeclareNode(NonArrayDK);
    $$->type = $1->type;
    $$->attr.name = savedName;
    $$->lineno = savedLineNo;
}
| type_spec id{
    $$ = newDeclareNode(ArrayDK);
    $$->type = $1->type;
    $$->attr.name = savedName;
    $$->lineno = savedLineNo;
}
| LBACE num RBACE SEMI{
    $$=$3;

    $$->child[0] = $5;
}
;

id : ID {
    savedName = copyString(tokenString);
    savedLineNo = lineno; }

num : NUM {
    $$ = newExpNode(ConstK);
    $$->lineno = lineno;
    $$->attr.val = atoi(tokenString); }

type_spec : INT{
    $$=newExpNode(IdK);
    $$->type = Integer;
    $$->lineno = lineno;
}
| VOID{$$=newExpNode(IdK);
    $$->type = Void;
    $$->lineno = lineno;};

```

```

func_declare : type_spec id {
    $$ = newDeclareNode(FuncDK);
    $$->type = $1->type;
    $$->attr.name = savedName;
    $$->lineno = savedLineNo;

    }LPAREN params RPAREN compound_stmt{
    $$ = $3;
    $$->child[0] = $5;
    $$->child[1] = $7;
}

;

params : param_list{$$=$1; }
|
VOID{$$=newDeclareNode(NonParamDK);
    $$->lineno = lineno;
    $$->type = Void;};

param_list : param_list COMMA param{
    YYSTYPE t = $1;
    if(t != NULL){
        while(t->sibling != NULL)
            t=t->sibling;
        t->sibling = $3;
        $$ = $1; }
    else $$ = $3;
}

;

param : param {$$=$1;};
param : type_spec id {
    $$ = newDeclareNode(ParamDK);
    $$->type = $1->type;
    $$->attr.name = savedName;
    $$->lineno = savedLineNo;
}
| type_spec id LBACE RBACE{
    $$ = newDeclareNode(ParamDK);
    $$->type = $1->type;
    $$->attr.name = savedName;
    $$->lineno = savedLineNo;
}
;

compound_stmt : LCURLY local_declare stmt_list RCURLY{
    $$=newStmtNode(CompK);
    $$->child[0] = $2;
    $$->child[1] = $3;
    $$->lineno = lineno;
};

local_declare : local_declare var_declare {
    YYSTYPE t = $1;
    if(t!=NULL){
        while(t->sibling != NULL)
            t=t->sibling;
        t->sibling = $2;
        $$ = $1;
    }
    else $$ = $2;
}
| {$$=NULL;};

```

```

stmt_list : stmt_list stmt
          | { YYSTYPE t = $1;
            if (t != NULL)
            { while (t->sibling != NULL)
                t = t->sibling;
              t->sibling = $2;
              $$ = $1;
            }
            else $$ = $2;
          }
          | { $$=NULL ;}
          ;

stmt : exp_stmt { $$ = $1; }
     | compound_stmt { $$ = $1; }
     | if_stmt { $$ = $1; }
     | if_else_stmt { $$=$1; }
     | iteration_stmt { $$ = $1; }
     | val_ret_stmt { $$ = $1; }
     | non_val_ret_stmt { $$ = $1; }
     ;

exp_stmt : exp SEMI{ $$=$1; } | SEMI{ $$=NULL; };
if_stmt : IF LPAREN exp RPAREN stmt %prec IFX{
        $$ = newStmtNode(IfK);
        $$->lineno = lineno;
        $$->child[0] = $3;
        $$->child[1] = $5;
      };

if_else_stmt : IF LPAREN exp RPAREN stmt ELSE stmt{
        $$ = newStmtNode(IfElseK);
        $$->lineno = lineno;
        $$->child[0] = $3;
        $$->child[1] = $5;
        $$->child[2] = $7;
      };

iteration_stmt : WHILE LPAREN exp RPAREN stmt{
        $$ = newStmtNode(IterK);
        $$->lineno = lineno;
        $$->child[0] = $3;
        $$->child[1] = $5;
      };

non_val_ret_stmt : RETURN SEMI{
        $$=newStmtNode(NonValRetK);
        $$->lineno = lineno;
      };

val_ret_stmt : RETURN exp SEMI{
        $$=newStmtNode(ValRetK);
        $$->lineno = lineno;
        $$->child[0] = $2;
      };

exp : var ASSIGN exp{
        $$=newExpNode(AssignK);
        $$->lineno = lineno;
        $$->child[0] = $1;
        $$->child[1] = $3;
      }
     | simple_exp{
        $$=$1;
      };

```

```

simple_exp : additive_exp relop additive_exp
          { $$ = $2;
            $$->child[0] = $1;
            $$->child[1] = $3;
          }
          |
          additive_exp { $$ = $1; }
          ;

var : id {
        $$ = newExpNode(IdK);
        $$->lineno = savedLineNo;
        $$->attr.name = savedName;
      }
     | id {
        $$ = newExpNode(IdK);
        $$->lineno = savedLineNo;
        $$->attr.name = savedName;
      } LBACE exp RBACE {
        $$=$2;
        $$->child[0] = $4;
      };

relop : GE { $$=newExpNode(BinK);
            $$->lineno = lineno;
            $$->attr.op=GE; }
       | GT { $$=newExpNode(BinK);
            $$->lineno = lineno;
            $$->attr.op=GT; }
       | LT { $$=newExpNode(BinK);
            $$->lineno = lineno;
            $$->attr.op=LT; }
       | LE { $$=newExpNode(BinK);
            $$->lineno = lineno;
            $$->attr.op=LE; }
       | EQ { $$=newExpNode(BinK);
            $$->lineno = lineno;
            $$->attr.op=EQ; }
       | NE { $$=newExpNode(BinK);
            $$->lineno = lineno;
            $$->attr.op=NE; } ;

additive_exp : additive_exp addop term{
        $$ = $2;
        $$->child[0] = $1;
        $$->child[1] = $3;
      }
           | term{
        $$=$1;
      };

addop : PLUS { $$=newExpNode(BinK);
              $$->lineno = lineno;
              $$->attr.op=PLUS; }
       | MINUS { $$=newExpNode(BinK);
              $$->lineno = lineno;
              $$->attr.op=MINUS; };

term : term mulop factor
      {
        $$ = $2;
        $$->child[0] = $1;
        $$->child[1] = $3;
      }
     | factor { $$ = $1; }
     ;

```

```

mulop      : TIMES{ $$=newExpNode(BinK);
               $$->lineno = lineno;
               $$->attr.op=TIMES;}
           | OVER{ $$=newExpNode(BinK);
               $$->lineno = lineno;
               $$->attr.op=OVER;};

call       : id{
               $$=newExpNode(CallK);
               $$->lineno = savedLineNo;
               $$->attr.name = savedName;

               } LPAREN args RPAREN{
               $$ = $2;
               $$->child[0] = $4;
               };

args       : arg_list{ $$=$1;} | { $$=NULL;};
arg_list   : arg_list COMMA exp{
               YYSTYPE t = $1;
               if(t!=NULL){
               while (t->sibling != NULL)
               |   t = t->sibling;
               t->sibling = $3;
               $$ = $1;
               }
               else $$=$3;

               } | exp { $$=$1;};

factor     : LPAREN exp RPAREN { $$ = $2; }
           | var{ $$=$1; } | call { $$=$1; }
           | num
           | ERROR { $$ = NULL; }
           ;

```

- 이후 rule 부분을 수정하여 Parser 를 구현하였습니다.

-program: 루트로 declare_list를 가지고 savedTree로 설정해 이것을 루트로 인식합니다

-declare_list: declare가 존재하지 않을 때까지 반복문을 돌면서 sibling 노드에 추가해줍니다

-declare: var 이나 fun 선언문으로 나뉩니다.

- var_declare: 배열 여부에 따라 노드를 생성해주고, 해당 노드에 타입과 id에서 저장해둔 savedName, savedLineNo를 가져와 저장합니다.

-id: 토큰 스트링을 읽어와 savedName에 저장하고, lineno 를 savedLineNo 에 저장합니다

-num: ConstK 타입 노드를 생성하고 현재 lineno, 토큰 스트링을 저장합니다.

-type_spec: IdK 타입 노드를 생성하고 각 타입과 lineno 를 저장합니다

-func_declare: Funck 타입 노드를 생성하고 id에서 저장해둔 이름, 라인 넘버를 추가하고 타입도 저장합니다.

매개변수 노드를 자신의 0번째 자식노드로 두고 복합문 노드를 자신의 1번째 노드로 둡니다.

-params : Void 만 존재할 경우 NonParamDK 타입의 선언문 노드를 새로 생성합니다. 아닌경우 param_list 를 생성합니다.

-param_list: param 없을 때까지 반복문을 돌며 sibling 에 추가합니다.

param: 선언을 통해 param타입을 생성합니다. 매개변수가 배열인 경우와 아닌 경우를 나눕니다.

-compound_stmt: Compk 타입 노드를 생성하고 local declare 를 0번 child 노드로 갖습니다. 1번 child 노드로 stmtlist 를 가집니다.

-local_declare과 stmt list 에서 문장이 나오지 않을 때까지 sibling 노드에 추가합니다.

-stmt는 각 타입에 맞게 stmtNode를 생성하여 child 노드로 연결 합니다.

-exp: var=exp 형식 또는 single_exp 형식으로 나뉘게 되며, 첫번째의 경우 AssignK 타입의 노드를 생성해서 var와 exp를 자식 노드로 연결합니다.

-relop: GT, GE, LT, LE, EQ, NE 노드를 생성하며 타입은 Bink 이고, attr->op 에 연산자를 저장합니다.

-addop, mulop 역시 비슷하게 구현합니다

실행결과 **cimnus_parser**

결과1. **Test.1.txt**

```
juwowa@juwowa: ~/loucomp_parse
Compound Statement:
  Call: function name = output
    Variable: name = x
    Variable: name = i
juwowa@juwowa:~/loucomp_parse$ ./cminus_parser test.1.txt

C-MINUS COMPILATION: ./test.1.txt

Syntax tree:
Function Declaration: name = gcd, return type = int
  Parameter: name = u, type = int
  Parameter: name = v, type = int
  Compound Statement:
    If-Else Statement:
      Op: ==
        Variable: name = v
        Const: 0
      Return Statement:
        Variable: name = u
      Return Statement:
        Call: function name = gcd
          Variable: name = v
          Op: -
            Variable: name = u
          Op: *
            Op: /
              Variable: name = u
              Variable: name = v
              Variable: name = v
Function Declaration: name = main, return type = void
  Void Parameter
  Compound Statement:
    Variable Declaration: name = x, type = int
    Variable Declaration: name = y, type = int
    Assign:
      Variable: name = x
      Call: function name = input
    Assign:
      Variable: name = y
      Call: function name = input
    Call: function name = output
    Call: function name = gcd
      Variable: name = x
      Variable: name = y
juwowa@juwowa:~/loucomp_parse$
```

결과2. Test.2.txt

```
juwowa@juwowa: ~/loucomp_parse
C-MINUS COMPILATION: ./test.2.txt

Syntax tree:
Function Declaration: name = main, return type = void
Void Parameter
Compound Statement:
  Variable Declaration: name = i, type = int
  Variable Declaration: name = x, type = int[]
  Const: 5
  Assign:
    Variable: name = i
    Const: 0
  While Statement:
    Op: <
    Variable: name = i
    Const: 5
    Compound Statement:
      Assign:
        Variable: name = x
        Variable: name = i
        Call: function name = input
      Assign:
        Variable: name = i
        Op: +
        Variable: name = i
        Const: 1
    Assign:
      Variable: name = i
      Const: 0
  While Statement:
    Op: <=
    Variable: name = i
    Const: 4
    Compound Statement:
      If Statement:
        Op: !=
        Variable: name = x
        Variable: name = i
        Const: 0
        Compound Statement:
          Call: function name = output
          Variable: name = x
          Variable: name = i
juwowa@juwowa:~/loucomp_parse$
```

결과3. Result 파일과 각각 diff 사용

```
juwowa@juwowa: ~/loucomp_parse
juwowa@juwowa:~/loucomp_parse$ diff <(. /cminus_parser test.2.txt ) result.2.txt
juwowa@juwowa:~/loucomp_parse$ diff <(. /cminus_parser test.1.txt ) result.1.txt
juwowa@juwowa:~/loucomp_parse$
```